TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN, ĐHQG-HCM

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO THỰC HÀNH HỆ ĐIỀU HÀNH

Lab 01

Thiết kế và đánh giá các tiện ích trên xv6-riscv

Xv6 User Programs: File System

Lớp	**Giảng viên hướng dẫn**
CQ2024/22	ThS. Lê Viết Long
	ThS. Phạm Tuấn Sơn

Nhóm thực hiện

Họ và tên	**Mã số sinh viên**
Phan Lê Đăng Khoa	24120347
Âu Bảo Trân	24120468
Dương Nhựt Duy	24120293

Thủ Đức, TP. Hồ Chí Minh

Mục lục

1 Giới thiệu **1**

1.1 Tổng quan và mục tiêu 1
1.2 Câu hỏi đánh giá và tiêu chí kiểm chứng 1
1.3 Tổ chức báo cáo 1
1.4 Môi trường triển khai và cấu hình máy 2
1.5 Kiến trúc chương trình và giao diện lời gọi hệ thống 2
1.6 Tích hợp và quy trình biên dịch 2

2 Thiết kế và triển khai **3**

2.1 `sleep`: chờ theo nhịp đồng hồ kernel 3
2.2 `pingpong`: trao đổi một byte hai chiều 3
2.3 `primes`: sàng nguyên tố bằng pipeline tiến trình 3
2.4 `cp`: sao chép nội dung tệp 4
2.5 `diff`: so sánh hai tệp theo vị trí dòng 4
2.6 `tree`: duyệt và biểu diễn cây thư mục 5
2.7 `du`: tổng hợp kích thước theo byte 5

3 Kiểm thử và đánh giá **6**

3.1 Phương pháp và khả năng tái lập 6
3.2 Ma trận kiểm thử và kết quả thực tế 6
3.3 Minh chứng tiêu biểu và phân tích kết quả 7
3.4 Thảo luận kết quả và phạm vi hiệu lực 7

4 Khó khăn và giới hạn **8**

5 Phân công và đóng góp **8**

6 Kết luận **9**

Tài liệu tham khảo **10**

Thiết kế và đánh giá các tiện ích trên xv6-riscv

Phan Lê Đăng Khoa Âu Bảo Trân Dương Nhựt Duy

Tóm tắt. Báo cáo trình bày thiết kế và kiểm chứng bảy tiện ích trong không gian người dùng trên xv6-riscv. Nhóm tiện ích quản lý tiến trình thực hiện chờ theo nhịp đồng hồ, trao đổi dữ liệu hai chiều và sàng số nguyên tố bằng chuỗi tiến trình nối qua pipe. Nhóm tiện ích thao tác với hệ thống tệp thực hiện sao chép, so sánh theo dòng, duyệt cây thư mục và tổng hợp kích thước tệp. Thực nghiệm trên QEMU sử dụng các kiểm thử MIT phù hợp với đề bài và các chương trình hỗ trợ chạy trực tiếp trong xv6. Kết quả đạt 5/5 kiểm thử MIT, 50/50 điểm bộ tiến trình mở rộng và 70/70 điểm bộ tích hợp; riêng năm nhóm kiểm thử chức năng và xử lý lỗi gồm 86 ca. Báo cáo phân tích cơ chế kết thúc, quản lý tài nguyên và giới hạn của từng giải pháp. Các điểm trên thuộc thang đánh giá nội bộ, không quy đổi thành điểm môn học.

Từ khóa: xv6-riscv; lời gọi hệ thống; pipe; hệ thống tệp; kiểm thử tự động.

1 Giới thiệu

1.1 Tổng quan và mục tiêu

Lab 01 tìm hiểu cách xây dựng chương trình trong không gian người dùng trên xv6-riscv thông qua bảy tiện ích: `sleep`, `pingpong`, `primes`, `cp`, `tree`, `du` và `diff`. Phạm vi chức năng được xác định theo đề `CQ_Lab01.docx`; tài liệu MIT cung cấp cơ sở đối chiếu cho ba tiện ích về tiến trình [1, 2].

Hai nhóm vấn đề chính gồm quản lý tiến trình và giao tiếp qua pipe (ống dẫn dữ liệu), cùng với thao tác trên hệ thống tệp. Mục tiêu là giải thích luồng dữ liệu, điều kiện kết thúc tệp (EOF) và cách quản lý bộ mô tả tệp (file descriptor, FD), đồng thời kiểm chứng kết quả bằng kiểm thử tự động. Tiêu chí đánh giá gồm tính đúng của đầu ra, mã thoát và khả năng xử lý lỗi tài nguyên.

1.2 Câu hỏi đánh giá và tiêu chí kiểm chứng

Nội dung thực nghiệm tập trung vào ba câu hỏi:

RQ1. Các tiến trình có thể trao đổi dữ liệu và kết thúc đúng thứ tự bằng pipe mà vẫn quản lý được FD và tiến trình con hay không?

RQ2. Các phép sao chép, so sánh và tổng hợp kích thước có đáp ứng quy ước đã xác định khi gặp EOF, tệp rỗng, dữ liệu nhị phân hoặc dòng dài hay không?

RQ3. Chương trình phản hồi như thế nào trước đối số sai, đường dẫn vượt giới hạn và tình huống thiếu tài nguyên?

Tính đúng được đánh giá bằng cách đối chiếu đầu ra và mã thoát với giá trị kỳ vọng, đồng thời kiểm tra chương trình hoàn tất trong thời gian cho phép. Thực nghiệm tập trung vào kiểm chứng chức năng, không đo lường hiệu năng.

1.3 Tổ chức báo cáo

Mục 2 mô tả thuật toán và quy tắc quản lý tài nguyên. Mục 3 trình bày phương pháp, ma trận kiểm thử và bằng chứng thực nghiệm. Mục 4 phân tích khó khăn, giới hạn và các điều kiện chưa được kiểm chứng.

1.4 Môi trường triển khai và cấu hình máy

Bảng 1 trình bày môi trường quan sát khi kiểm thử ngày 15/10/2026. Dung lượng RAM trong bảng là tài nguyên được WSL2 cấp cho Linux. Cấu hình công cụ được ghi nhận để tái lập thí nghiệm theo hướng dẫn môi trường xv6 [3].

Bảng 1. Môi trường thực nghiệm; cấu hình máy giả lập lấy từ Makefile và kernel.

Thành phần	Cấu hình đã xác minh
Hệ điều hành chủ	Ubuntu 24.04.5 LTS trên WSL2; kernel Linux 6.6.87.2; kiến trúc x86_64.
CPU và bộ nhớ khả dụng	AMD Ryzen 5 5600H; 6 nhân/12 luồng được WSL2 nhận diện; khoảng 15 GiB RAM được cấp cho Linux.
Công cụ	RISC-V GCC 13.3.0; QEMU 8.2.2; Python 3.12.3; GNU Make.
Máy giả lập xv6	RISC-V `virt`, 128 MiB RAM, 3 CPU ảo; đĩa `fs.img` qua virtio-blk.
Bản mã nguồn	Nhánh `util`, commit `35ba531`; bản gốc `79e8024`.
Giới hạn kernel	`NPROC=64`, `NOFILE=16`, `MAXPATH=128`; tên mục thư mục tối đa `DIRSIZ=14` byte.

1.5 Kiến trúc chương trình và giao diện lời gọi hệ thống

Shell thực thi các tiện ích thông qua `fork`/`exec`; phần xử lý nằm trong không gian người dùng. Nhóm tiến trình sử dụng `pipe`, `read`, `write`, `wait` và đồng hồ kernel. Nhóm thao tác tệp sử dụng `open`, `fstat`, `read`, `write` và `close`. Thư viện người dùng cung cấp các hàm xử lý chuỗi, cấp phát bộ nhớ và định dạng; lời gọi hệ thống chuyển yêu cầu đến kernel. Cách phân lớp này cho phép bổ sung tiện ích trên giao diện kernel có sẵn.

1.6 Tích hợp và quy trình biên dịch

Bảy chương trình có tệp C tương ứng trong `user/` và được thêm vào `UPROGS`. Makefile liên kết chúng với thư viện người dùng của xv6, sau đó `mkfs` đóng gói vào `fs.img`; `make qemu` khởi động kernel và shell. Phần triển khai giữ nguyên kernel và bộ kiểm thử MIT.

Bản kiểm thử được xuất từ commit `35ba531` sang thư mục tạm bằng `git archive HEAD`. Tại bản sao này, nhóm chạy ba bước:

```
make clean
make -j4 kernel/kernel fs.img
make check-lab01
```

Thư mục riêng giúp cô lập dữ liệu và các tệp sinh ra khi kiểm thử. Mã C được biên dịch với `-Wall -Werror`; nhật ký và định danh commit được lưu cùng kết quả để truy vết [5].

2 Thiết kế và triển khai

2.1 `sleep`: chờ theo nhịp đồng hồ kernel

Mục tiêu. `sleep ticks` tạm dừng tiến trình theo số nhịp đồng hồ (tick) của kernel. Giá trị đầu vào được hiểu theo đơn vị tick, không quy đổi trực tiếp thành giây.

Cách tiếp cận. Chương trình nhận đúng một đối số. Trước khi gọi `atoi`, chương trình kiểm tra chuỗi không rỗng, chỉ gồm chữ số và biểu diễn giá trị trong miền $0 \dots 2147483647$. Kiểm tra trước phép nhân và cộng giúp tránh tràn số khi phân tích đối số. Với dữ liệu hợp lệ, chương trình gọi `sleep(ticks)` và kết thúc bằng `exit(0)`.

Trường hợp biên. Đối số bị thiếu hoặc dư, số âm, ký tự không hợp lệ và giá trị vượt miền biểu diễn đều được báo qua `stderr`, kèm mã thoát 1. `sleep 0` là hợp lệ. Với `sleep 10`, khoảng thời gian đo bằng `uptime()` phải đạt ít nhất 10 tick; lịch thực thi có thể làm giá trị quan sát lớn hơn yêu cầu.

2.2 `pingpong`: trao đổi một byte hai chiều

Mục tiêu. Tiến trình cha gửi một byte cho con. Tiến trình con in mã định danh tiến trình (PID) cùng `received ping`, rồi gửi lại một byte; cha nhận phản hồi và in PID cùng `received pong`.

Cách tiếp cận. Hai pipe một chiều được tạo trước `fork`: `tochild` truyền từ cha đến con, `toparent` truyền theo chiều ngược lại. Mỗi bên đóng ngay hai đầu không sử dụng. Con in ping trước khi gửi phản hồi; cha chỉ in pong sau khi đọc thành công. Sự phụ thuộc dữ liệu này bảo đảm thứ tự ping trước pong.

Luồng giao tiếp

$$\text{Cha: ghi 1 byte} \xrightarrow{\texttt{tochild}} \text{Con: đọc, in ping}$$
$$\text{Cha: đọc, in pong} \xleftarrow{\texttt{toparent}} \text{Con: ghi trả 1 byte}$$

Quản lý tài nguyên. Kết quả mỗi lần `read`/`write` được kiểm tra để bảo đảm truyền đủ một byte. Sau trao đổi, hai bên đóng các đầu pipe; cha gọi `wait` và kiểm tra trạng thái con. Nếu tạo pipe hoặc `fork` thất bại, các FD đã mở được đóng. Mã thoát là 0 khi thành công và 1 khi có lỗi.

2.3 `primes`: sàng nguyên tố bằng pipeline tiến trình

Mục tiêu và thuật toán. Tiến trình phát gửi các số nguyên từ 2 đến 280, mỗi số 4 byte. Tầng lọc đọc số đầu làm nguyên tố p, in `prime p`, loại các bội của p và chuyển phần còn lại sang tầng sau. Số đến đầu tiên của tầng kế tiếp không có ước nguyên tố nhỏ hơn, nên là số nguyên tố tiếp theo.

Tạo tầng khi cần. Chỉ khi tìm thấy số không bị loại, tầng hiện tại mới tạo pipe và gọi `fork`. Tiến trình con thay FD đầu vào rồi tiếp tục vòng lặp. Mỗi nguyên tố có một tiến trình riêng; cách triển khai này không làm tăng độ sâu ngăn xếp do lời gọi đệ quy. Kết quả kỳ vọng gồm 59 số, từ 2 đến 277.

Kết thúc và tài nguyên. Mỗi tầng đóng đầu ghi trước khi gọi `wait`, cho phép tầng sau nhận EOF sau khi đọc hết dữ liệu còn lại. Chuỗi chờ con truyền trạng thái lỗi về tiến trình phát và bảo đảm tiến trình chính chỉ thoát khi toàn bộ pipeline kết thúc. Phạm vi 2–280 tuân theo đề bài và giới hạn tài nguyên xv6.

2.4 `cp`: sao chép nội dung tệp

Mục tiêu. `cp src dst` sao chép tệp thông thường, tạo tệp đích mới hoặc ghi đè nội dung đã tồn tại.

Cách tiếp cận. Chương trình mở nguồn bằng `O_RDONLY` và kiểm tra loại tệp qua `fstat`. Nếu đích tồn tại, loại tệp và cặp `dev`/`ino` được đối chiếu trước khi mở đích bằng `O_WRONLY | O_CREATE | O_TRUNC`. Cặp định danh này nhận diện được hai đường dẫn cùng tham chiếu một tệp qua liên kết cứng (hard link), giúp tránh cắt ngắn chính tệp nguồn.

Dữ liệu được đọc theo bộ đệm 1024 byte. Với mỗi khối *n* byte, chương trình ghi lặp đến khi đủ *n* byte rồi mới đọc tiếp, thay vì giả định một lần `write` luôn hoàn tất yêu cầu. Quá trình sao chép dừng khi gặp EOF; các FD đã mở được đóng trước khi kết thúc.

Trường hợp biên và giới hạn. `O_TRUNC` loại phần đuôi cũ khi ghi đè tệp dài bằng tệp ngắn. Tệp rỗng, dữ liệu nhị phân và tệp lớn hơn bộ đệm được xử lý theo số byte thực tế. Chương trình trả 0 khi thành công; lỗi đối số, loại tệp hoặc thao tác vào/ra có thông báo và mã thoát 1. Lỗi ghi giữa chừng có thể để lại tệp đích chưa đầy đủ; bản triển khai chưa hỗ trợ sao chép nguyên tử hoặc khôi phục nội dung cũ.

2.5 `diff`: so sánh hai tệp theo vị trí dòng

Mục tiêu. `diff file1 file2 [-q]` so sánh các dòng cùng vị trí. Hai tệp giống nhau không tạo đầu ra; khác biệt được đánh dấu `<` cho tệp thứ nhất và `>` cho tệp thứ hai. Tùy chọn `-q` có thể đứng trước hoặc sau hai tên tệp.

Đọc và so sánh. Hàm `read_line` đọc đến ký tự xuống dòng hoặc EOF, trả cả độ dài và nội dung. Bộ đệm ban đầu 128 byte được tăng gấp đôi khi cần. So sánh độ dài và từng byte giúp phân biệt ký tự xuống dòng cuối, đồng thời xử lý byte NUL (giá trị 0) bên trong dữ liệu. Hai bộ đệm được tái sử dụng rồi giải phóng khi kết thúc.

Quy ước đầu ra. Chương trình in `< tệp:dòng: nội-dung` hoặc `> tệp:dòng: nội-dung`. Khi một tệp kết thúc trước, chỉ các dòng còn lại của tệp kia được in, không bổ sung dòng báo EOF. Ví dụ, với `f1` chứa một dòng và `f2` có thêm dòng `two`, kết quả chạy thật là:

```
$ diff f1 f2
> f2:2: two
```

Vị trí ký hiệu </> và cách biểu diễn EOF khác ví dụ định dạng trong đề [1]. Dòng cuối thiếu ký tự xuống dòng được báo bằng `\ No newline at end of file`. Bộ kiểm thử nội bộ đối chiếu các quy ước này; kết quả đạt chưa xác nhận yêu cầu định dạng của bộ chấm giảng viên.

Chế độ yên lặng và lỗi. Với `-q`, chương trình dừng tại khác biệt đầu tiên và chỉ in `diff: files differ`. Mã thoát là 0 nếu bằng nhau, 1 nếu khác và 2 nếu có lỗi. Lỗi mở, loại tệp, đọc hoặc cấp phát được báo rõ; các FD và bộ nhớ đã cấp phát được thu hồi.

Giới hạn thuật toán. Chương trình ghép dòng theo số thứ tự, không tìm dãy con chung dài nhất để tối ưu phép chèn hoặc xóa. Thời gian xử lý tuyến tính theo lượng dữ liệu đọc; bộ nhớ tỉ lệ với hai dòng dài nhất. Đọc từng byte tạo nhiều lời gọi hệ thống với tệp lớn; độ dài dòng vẫn chịu giới hạn bộ nhớ và ngưỡng cấp phát.

2.6 `tree`: duyệt và biểu diễn cây thư mục

Mục tiêu. `tree [path] [-L depth] [-d]` in cấu trúc phân cấp. Đường dẫn mặc định là `.`; `-L` giới hạn độ sâu, `-d` chỉ hiển thị thư mục.

Cách tiếp cận. Mỗi thư mục được mở chỉ đọc, kiểm tra bằng `fstat` và duyệt các `struct dirent` qua `read`. Chương trình bỏ mục không sử dụng, `.` và `..`, sau đó gọi `stat` để lọc loại tệp và quyết định đệ quy. Ký hiệu hiển thị thống nhất là `|--` và `|  `; thứ tự theo các mục trên đĩa, không sắp xếp theo tên.

Độ sâu hiển thị được tính từ gốc là 0, trong khi biến mức của hàm bắt đầu từ 1. Vì vậy, `-L 0` chỉ in gốc, `-L 1` in thêm các mục con trực tiếp. Tùy chọn `-d` chỉ giữ thư mục trong đầu ra. Chương trình nhận tối đa một đường dẫn; tùy chọn có thể đứng trước hoặc sau đường dẫn.

An toàn dữ liệu và lỗi. Trường `dirent.name` có 14 byte và có thể thiếu NUL kết thúc. Tên được chép sang bộ đệm 15 byte rồi thêm NUL trước khi xử lý bằng hàm chuỗi. Đường dẫn gốc và đường dẫn ghép dài tối đa 127 byte, chừa một byte NUL trong `MAXPATH=128`. Lỗi mở, đọc, `stat` hoặc duyệt nhánh con được truyền về gốc; chương trình trả mã 1 khi có lỗi và đóng FD trước khi rời mỗi tầng.

2.7 `du`: tổng hợp kích thước theo byte

Mục tiêu và quy ước. `du [path] [-a] [-s]` tính tổng kích thước logic của tệp theo `st.size`, đơn vị byte. Phép tính không cộng kích thước bản thân thư mục và không đo số khối đĩa cấp phát. Thư mục rỗng và thiết bị được tính 0 byte. Quy ước bỏ kích thước bản thân thư mục được đề cho phép nếu trình bày nhất quán [1].

Với thư mục d, giá trị được tính hậu thứ tự:

$$S(d) = \sum_{x \in \mathrm{con}(d)} S(x), \qquad S(f) = \mathrm{st.size}(f) \quad \text{với tệp thông thường } f.$$

Cách tiếp cận. Chương trình duyệt các mục và nối đường dẫn như `tree`; tính từng cây con, cộng vào tổng `uint64`, sau đó mới in thư mục cha. Trạng thái lỗi được trả riêng với biến tổng để phân biệt lỗi mở với thư mục rỗng. Hàm định dạng thập phân riêng chuyển số 64 bit thành chuỗi, do `printf` của bản xv6 này vẫn thu hẹp giá trị qua `printint(int)` khi dùng định dạng dài.

Quy tắc hiển thị. Mặc định chỉ in thư mục; `-a` in thêm tệp; `-s` chỉ in tổng tại gốc và được ưu tiên khi kết hợp với `-a`. Nếu gốc là tệp, chỉ `-a` hoặc `-s` mới tạo đầu ra. Kết quả có dạng `bytes\tpath`. Khi duyệt gặp lỗi, chế độ `-s` không xuất tổng chưa đầy đủ và chương trình trả mã thoát 1.

Giới hạn chung. Hai chương trình giữ một FD cho mỗi thư mục tổ tiên; cây sâu có thể chạm giới hạn `NOFILE=16`. Khi đó, chương trình báo lỗi và trả trạng thái thất bại. Phép tính của `du` cộng theo từng mục thư mục, nên các liên kết cứng đến cùng inode có thể được tính nhiều lần.

3 Kiểm thử và đánh giá

3.1 Phương pháp và khả năng tái lập

Kiểm thử ngày 15/10/2026 được chạy lại trên bản xuất sạch của commit `35ba531`. Lệnh `make check-lab01` gọi bộ MIT cho ba tiện ích tiến trình, bộ TV1 mở rộng và bộ tích hợp. Bộ MIT có ca xác minh `sys_sleep`; TV1 lặp `pingpong` 50 lần và `primes` 20 lần.

Các chương trình hỗ trợ bằng C chạy thật trong xv6 qua QEMU, tạo dữ liệu, thu đầu ra và gọi `wait(&status)` để kiểm tra mã thoát. Từng nhóm tích hợp tạo lại `fs.img` để cô lập dữ liệu; các chương trình hỗ trợ chỉ có trong ảnh hệ thống tệp dùng để kiểm thử.

3.2 Ma trận kiểm thử và kết quả thực tế

Bảng 2. Kết quả kiểm thử; 86 ca thuộc năm nhóm chức năng và xử lý lỗi.

Nhóm	Nội dung được đối chiếu tự động	Kết quả
Biên dịch	Kernel, hệ thống tệp, bảy tiện ích; `-Wall -Werror`.	Đạt
MIT	3 ca `sleep`, 1 ca `pingpong`, 1 ca `primes`.	5/5 ca
TV1 mở rộng	Đối số, tick, mã thoát, kiểm thử lặp và pipeline.	50/50 điểm
`cp`	Tệp rỗng, nhị phân 8193 byte, ghi đè, cùng inode; lỗi.	13/13 ca
`diff`	EOF, `-q`, xuống dòng, NUL, dòng 4098 byte; lỗi.	23/23 ca
`tree`	`-L 0/1/2`, `-d`, tên 14 byte; lỗi đường dẫn và FD.	21/21 ca
`du`	Tổng byte, `-a/-s`, tệp gốc, rỗng; lỗi đường dẫn và FD.	17/17 ca
Tài nguyên	Cạn FD khi mở tệp, tạo pipe và tạo tầng sàng.	12/12 ca
Định dạng số	Hàm in của `du`: 0, 2^{32} và $2^{64} - 1$.	3/3 giá trị
Tích hợp	Luồng shell sử dụng đủ bảy tiện ích.	Đạt

Cả ba bộ kết thúc với mã 0. Hình 1 trình bày các dòng kết quả được lọc từ nhật ký lần chạy lại; điểm 50/50 và 70/70 thuộc thang nội bộ, không phải điểm môn học [5].

TEST RESULTS | filtered log excerpt

```
./grade-lab-util sleep pingpong primes
./grade-tv1
Score: 50/50
./grade-lab01
LAB01_CP_OK: 13 cases
LAB01_DIFF_OK: 23 cases
LAB01_TREE_OK: 21 cases
LAB01_DU_OK: 17 cases
LAB01_RESOURCES_OK: 12 cases
Score: 70/70
```

Hình 1. Trích đoạn tổng kết kiểm thử, dựng từ nhật ký đã lọc trên commit `35ba531`.

3.3 Minh chứng tiêu biểu và phân tích kết quả

Hình 2 trình bày trích đoạn liên tục từ nhật ký QEMU của luồng tích hợp. Hai tệp cùng chứa `hello` và ký tự xuống dòng, mỗi tệp 6 byte; `diff -q` không tạo đầu ra và `du demo -s` trả tổng 12 byte.

xv6 / QEMU | continuous log excerpt

```
$ mkdir demo
$ echo hello > demo/a
$ cp demo/a demo/b
$ diff -q demo/a demo/b
$ tree demo -L 1
demo
|-- a
|-- b
$ du demo -s
12      demo
$ sleep 0
$ pingpong
11: received ping
10: received pong
$ primes | wc
59 118 561
```

Hình 2. Luồng tích hợp trong xv6, dựng từ nhật ký QEMU; `primes | wc` ghi nhận 59 dòng.

Dữ liệu lớn hơn bộ đệm. Nội dung nhị phân 8193 byte của `cp` được đối chiếu toàn bộ, gồm khối cuối không đầy. Với `diff`, dòng 4098 byte có thay đổi sau byte 4096 giúp phát hiện lỗi cắt ngắn; dữ liệu chứa NUL kiểm tra phép so sánh theo độ dài.

Định dạng 64 bit. Hàm in của `du` được gọi trực tiếp với 0, 2^{32} và $2^{64} - 1$. Kết quả đúng xác minh định dạng số, không chứng minh xv6 đã tạo được tệp có kích thước tương ứng.

3.4 Thảo luận kết quả và phạm vi hiệu lực

RQ1 – Tiến trình. Đầu ra của mỗi lần chạy `primes` được đối chiếu với 59 số nguyên tố xác định bằng phép chia thử độc lập. Với `pingpong`, 100 thông báo đúng thứ tự có PID khác nhau trong từng cặp. Cả hai trở lại shell và chạy được trong pipeline. Phép đo `sleep 10` đạt ít nhất 10 tick.

RQ2 – Dữ liệu. Các ca tệp rỗng, nhị phân, ghi đè, dòng dài và NUL cho thấy phép xử lý theo byte đáp ứng các trường hợp đã thử. Tổng của `du` phù hợp với quy ước kích thước logic.

RQ3 – Lỗi. Các ca kiểm thử chủ động dùng hết FD để đối chiếu thông báo và mã thoát; giới hạn thời gian giúp phát hiện treo. Duyệt cây 20 cấp thất bại có kiểm soát, còn `tree -L 2` thành công khi giới hạn độ sâu.

Phạm vi hiệu lực. Chưa chủ động gây ra mọi lỗi `fork`, cạn bộ nhớ, đầy đĩa hoặc thay đổi hệ thống tệp đồng thời. Bộ kiểm thử chưa vét cạn. Thời gian chạy QEMU/WSL2 chịu ảnh hưởng lịch thực thi máy chủ, nên không được dùng để suy luận hiệu năng.

4 Khó khăn và giới hạn

Thông báo lỗi và mã thoát không nhất quán. Nhật ký trước tích hợp ghi nhận `tree` và `du` báo lỗi đường dẫn nhưng vẫn gọi `exit(0)`. Nhóm sửa bằng cách truyền trạng thái từ hàm duyệt về `main`. Với `du`, tách trạng thái lỗi khỏi biến tổng còn giúp chế độ `-s` tránh xuất kết quả chưa đầy đủ. Các ca kiểm thử hồi quy cho đường dẫn lỗi đều đạt.

Tên mục thư mục có thể thiếu ký tự kết thúc chuỗi. Tên dài đúng `DIRSIZ` có thể thiếu NUL kết thúc; dùng trực tiếp với hàm chuỗi có nguy cơ đọc vượt vùng nhớ. Bản sửa sử dụng bộ đệm `DIRSIZ+1`, thêm NUL và kiểm tra độ dài trước khi nối đường dẫn. Các ca tên 14 byte và đường dẫn vượt `MAXPATH` kiểm chứng hai điều kiện này.

Kiểu dữ liệu và hàm in không đồng nhất. Tổng của `du` dùng `uint64`, nhưng đổi định dạng sang `%lu` chưa giải quyết được việc `printf` chuyển giá trị vào hàm nhận `int`. Hàm chuyển thập phân riêng tránh thu hẹp kiểu. Các ca giá trị lớn ở Mục 3 giúp phát hiện hồi quy của lỗi này.

EOF và giới hạn tài nguyên. Giữ đầu ghi pipe không cần thiết có thể khiến tầng sau chờ dữ liệu vô hạn. Vì vậy, đóng FD phải đi cùng thiết kế đồng bộ tiến trình. Với duyệt thư mục, bản triển khai vẫn giữ FD của các thư mục tổ tiên và báo lỗi khi vượt giới hạn. Các ca cạn FD kiểm tra khả năng thất bại có kiểm soát; chúng chưa chứng minh xử lý được mọi tình huống thiếu tài nguyên.

5 Phân công và đóng góp

Bảng 3 tổng hợp phần mã nguồn theo hồ sơ bàn giao và trách nhiệm báo cáo theo phân công của nhóm [4]. Hồ sơ commit xác định phần triển khai; bảng không đánh giá định lượng mức đóng góp cá nhân.

Bảng 3. Phân công triển khai và tổng hợp báo cáo.

Thành viên	Phần triển khai và phần báo cáo được giao
Phan Lê Đăng Khoa	`sleep`, `pingpong`, `primes`; nhánh `lab01-process`, bàn giao `a577dd5`. Phân công báo cáo: phần tiến trình, điều phối và rà soát chung.
Âu Bảo Trân	`cp`, `diff`; nhánh `lab01-files`, bàn giao `bf5cc3a`. Phân công báo cáo: tổng hợp, định dạng, bìa và kết luận.
Dương Nhựt Duy	`tree`, `du`; nhánh `lab01-directories`, bàn giao `6d9d56a`. Phân công báo cáo: môi trường, kiểm thử và tổng hợp khó khăn.

Khả năng tái lập. Mã nguồn được xác định bằng commit; các script, chương trình hỗ trợ và nhật ký cho phép đối chiếu từng nhóm kết quả. Minh chứng trong Hình 1 và Hình 2 được dựng trực tiếp từ nhật ký lần chạy lại, với cách trích xuất lưu cùng bằng chứng [4, 5].

6 Kết luận

Lab 01 đã triển khai đủ bảy tiện ích trên xv6-riscv và chạy lại thành công các bộ kiểm thử được lựa chọn trong phạm vi bài làm. Kết quả bao phủ các trường hợp thông thường cùng những tình huống biên liên quan đến EOF, dữ liệu lớn hơn bộ đệm, hai đường dẫn cùng trỏ đến một inode, tên tệp dài 14 byte và thiếu FD.

Bài thực hành làm rõ mối liên hệ giữa thuật toán và tài nguyên hệ điều hành. Đóng đúng đầu pipe cho phép bên đọc nhận EOF; chuỗi lời gọi `wait` bảo đảm tiến trình phát chỉ thoát sau khi toàn bộ pipeline kết thúc. Thông tin loại tệp và inode hỗ trợ duyệt thư mục và tránh tự ghi đè nguồn. Việc đối chiếu mã thoát cùng dữ liệu kỳ vọng độc lập giúp phát hiện lỗi khó nhận ra qua quan sát đầu ra đơn thuần.

Các giới hạn còn lại gồm phạm vi sàng 2–280, số FD khi duyệt cây sâu, việc tính liên kết cứng theo từng mục thư mục và phép so sánh theo vị trí dòng. Định dạng `diff` cần được đối chiếu thêm nếu bộ chấm yêu cầu đầu ra trùng ví dụ của đề. Các hướng phát triển tiếp theo là đọc dòng theo khối, giảm FD giữ khi duyệt cây, bổ sung kiểm thử lỗi cấp phát và ghi đĩa, cũng như cơ chế bảo toàn tệp đích khi sao chép thất bại.

Tài liệu tham khảo

[1] Đề bài môn Hệ điều hành, *Lab 01 – Xv6 User Programs: File System*, tài liệu `CQ_Lab01.docx`. Phạm vi chức năng, tiêu chí đánh giá và quy định nộp bài.

[2] MIT, *Lab: Xv6 and Unix utilities*, học phần 6.1810, 2024. Truy cập ngày 15/10/2026. `https://pdos.csail.mit.edu/6.1810/2024/labs/util.html`

[3] MIT, *Tools*, học phần 6.1810, 2024. Truy cập ngày 15/10/2026. `https://pdos.csail.mit.edu/6.1810/2024/tools.html`

[4] Nhóm thực hiện, *Mã nguồn và hồ sơ bàn giao Lab 01*, commit `35ba531`; các thư mục `user/`, `tests/`, `docs/lab01/` và `docs/tv1/`. `https://github.com/EndlessMelody/os-lab1`

[5] Nhóm thực hiện, *Nhật ký kiểm thử Lab 01 ngày 15/10/2026*, thư mục `docs/report-validation/`: môi trường và hồ sơ kiểm chứng; thư mục con `upgrade/`: nhật ký biên dịch, kiểm thử, QEMU và ví dụ `diff`; thư mục con `figures/`: minh chứng cùng nội dung trích xuất. Công cụ dựng hình: `tools/render-report-evidence.py`.